

Convolutional Neural Networks

Jonghyun Choi

Associate Professor

Department of Electrical and Computer Engineering
Seoul National University



Schedule

Each hour is scheduled as **50 min lecture + 10 min break**

- **Day 1:** 4 hr lecture + 4 hr practice

- **AM:** CNN, Transformers, Object detection
- **PM:** DETR, MI-DETR

- **Day 2:** 4 hr lecture + 4 hr practice

- **AM:** Self-supervised learning, Weakly supervised learning, Multimodal models, Semantic segmentation
- **PM:** DINO, SAM variants

Computer Vision Tasks

Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Objects

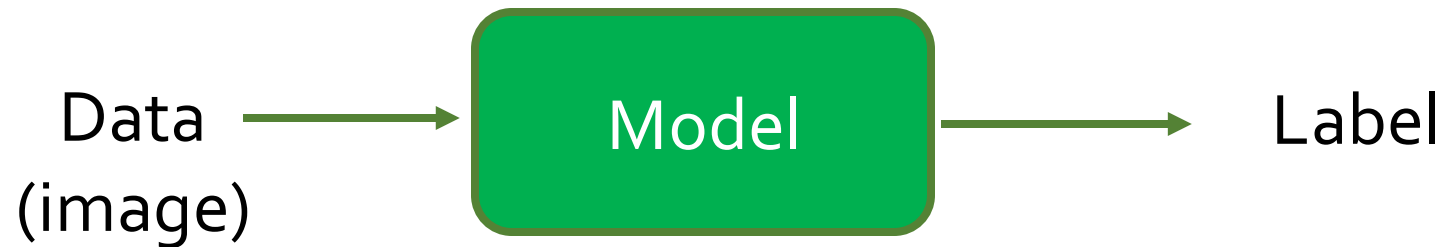
Instance Segmentation



DOG, DOG, CAT

Why is the image classification the first example of machine learning?

- Simplest ML setting



- MLP is too old and CNN is easy to try and looks new!
 - MLP (1960)
 - CNN (1990, 2012 with renaissance)

Mapping pixels to label is very difficult



How to recognize objects?

image



filter


$$\times \quad / n = 1.000$$

of pixels

image

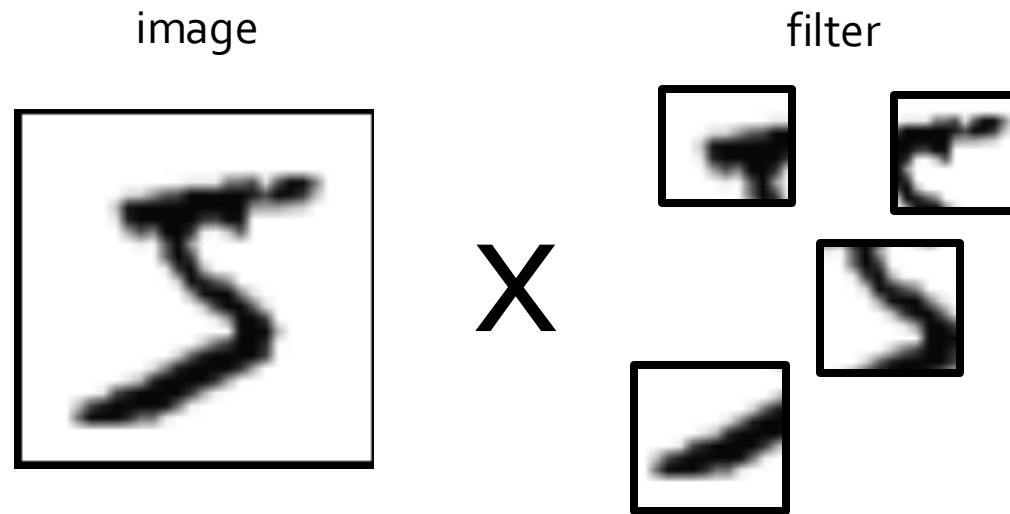


filter


$$\times \quad / n = 0.321$$

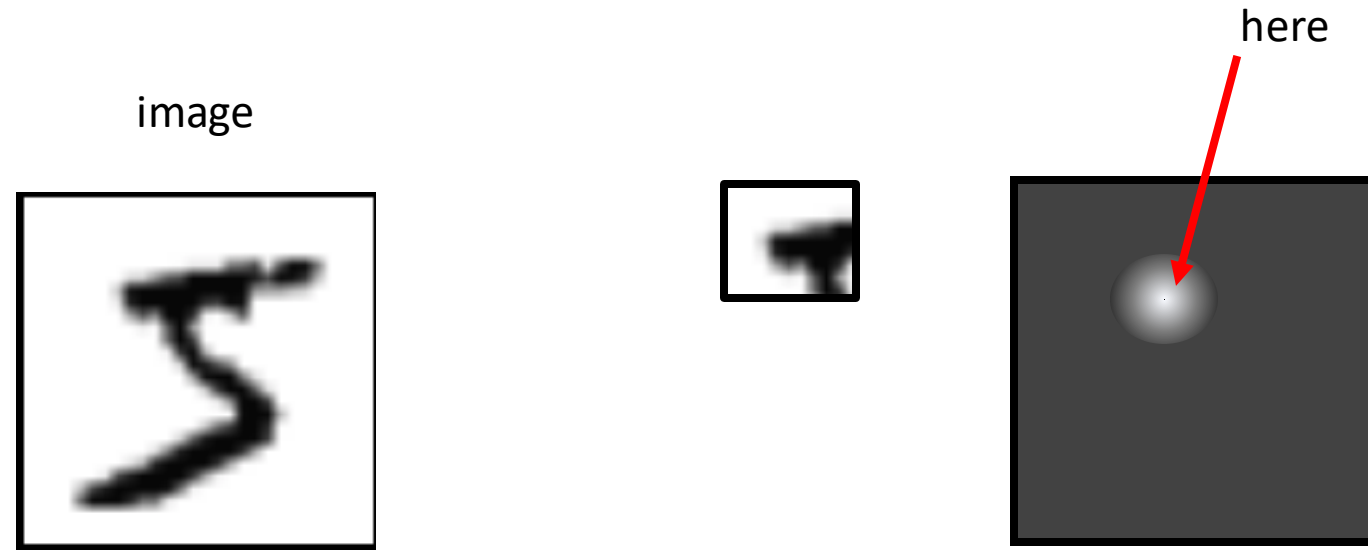
Many filters are needed!

What if we break the filter into meaningful pieces?



Where to multiply?

What about scanning the filter and get the
best response

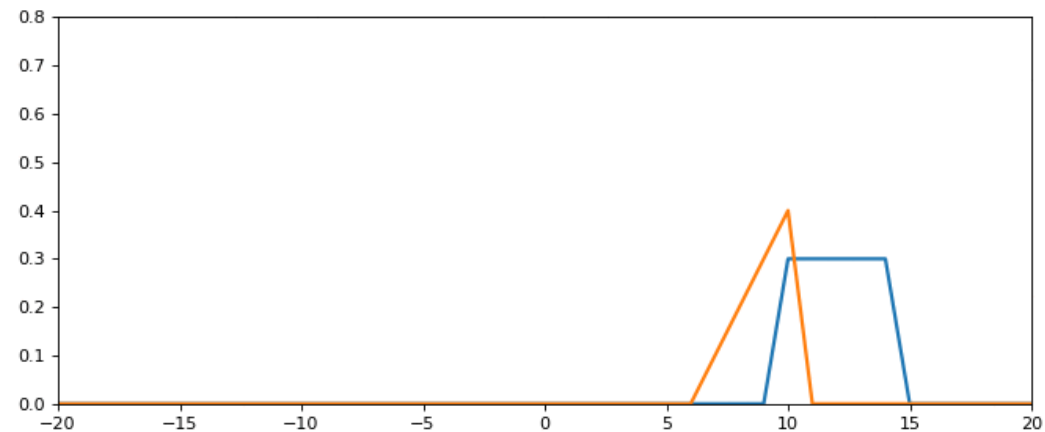


Take the max output!

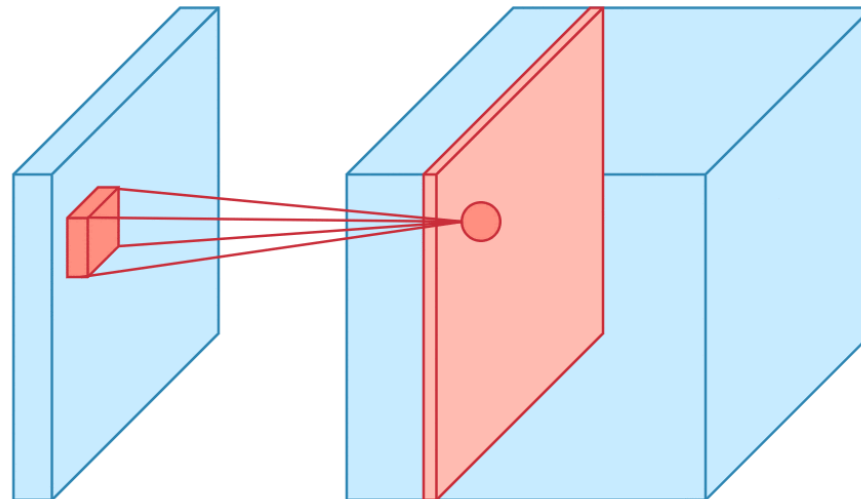
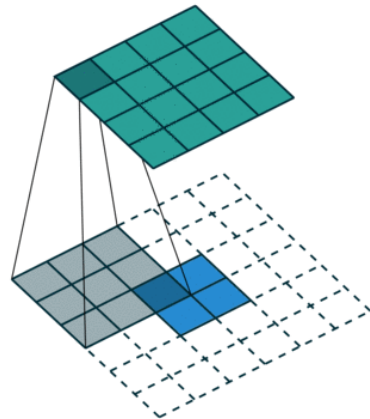
Convolution

- A mathematical operation on two functions to produce a third function
- Uses symbol: $*$

1-D convolution



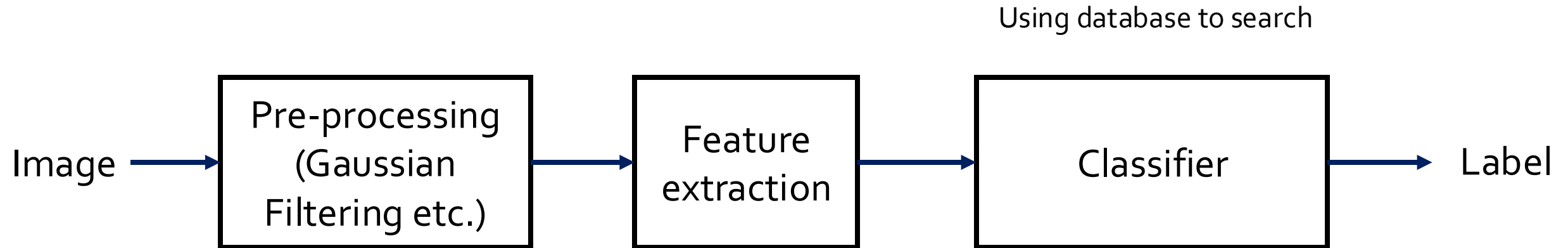
2-D convolution



Evaluation

- Accuracy = $\frac{\text{\# correctly classified samples}}{\text{\# total samples in test set}}$

Long-lived pipeline for image classification



Then 2012 comes

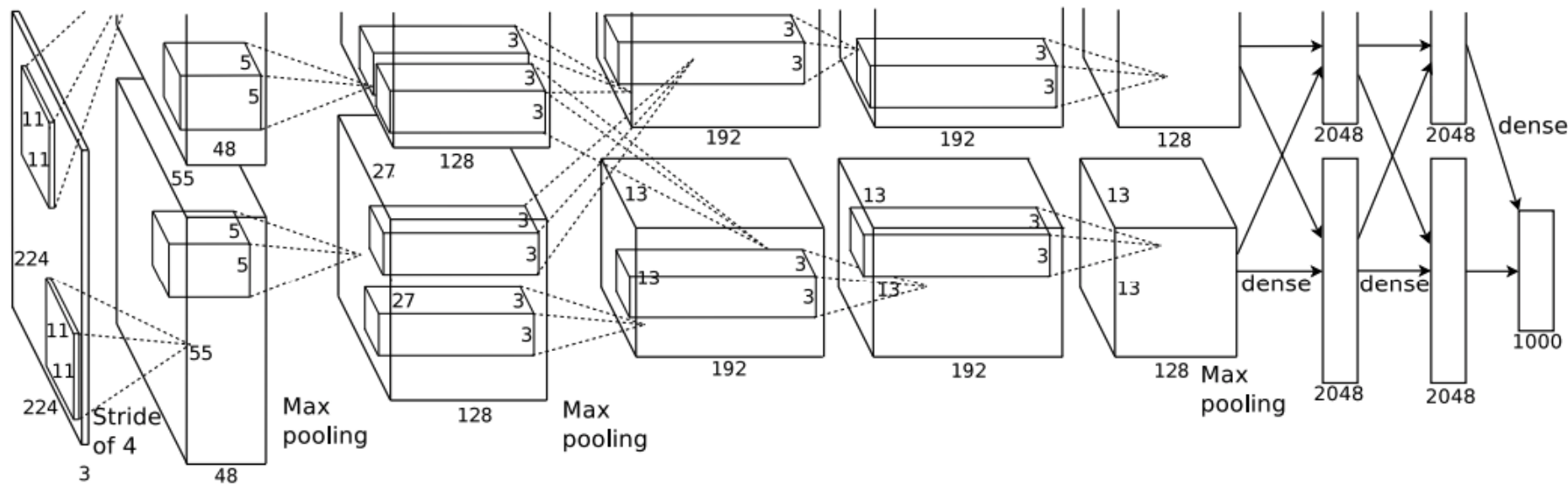
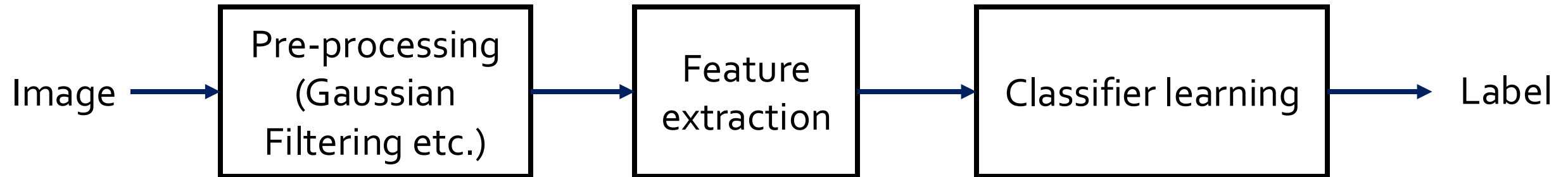


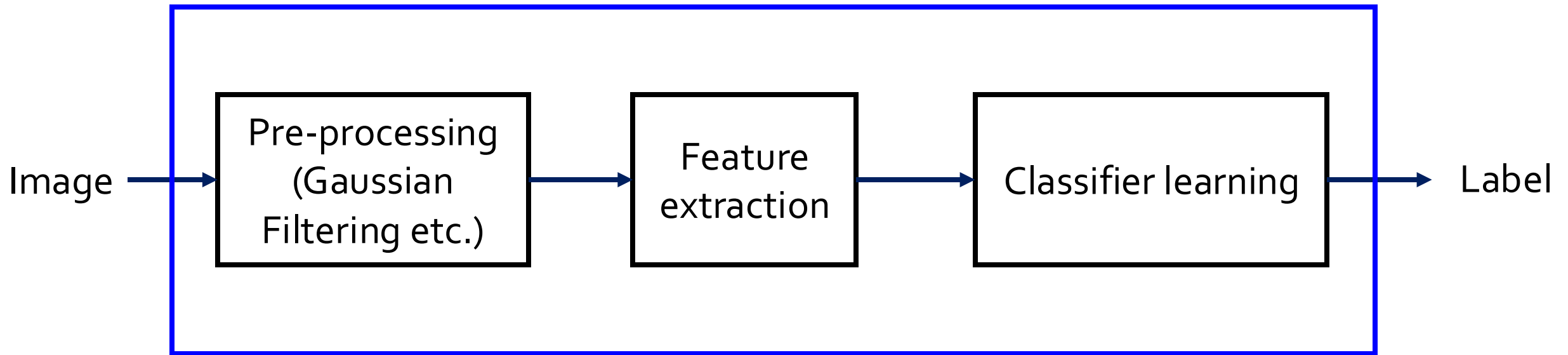
Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

Long-lived pipeline for image classification



Long-lived pipeline for image classification

CNN



End-to-end image classification learning

AlexNet

Named by later researchers

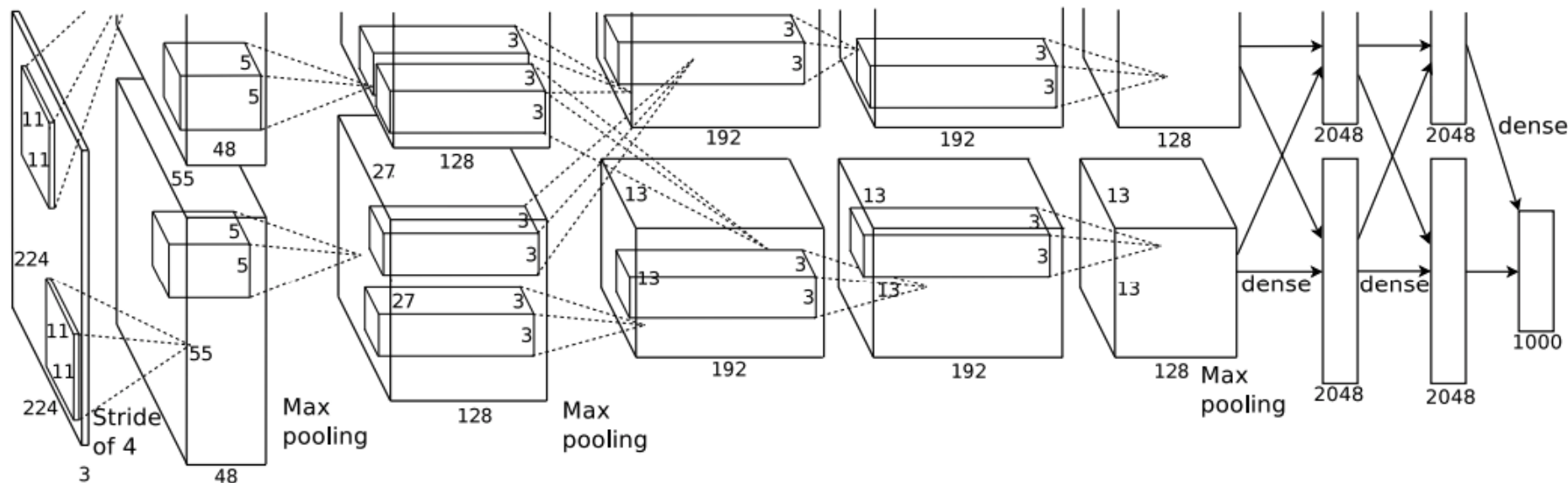
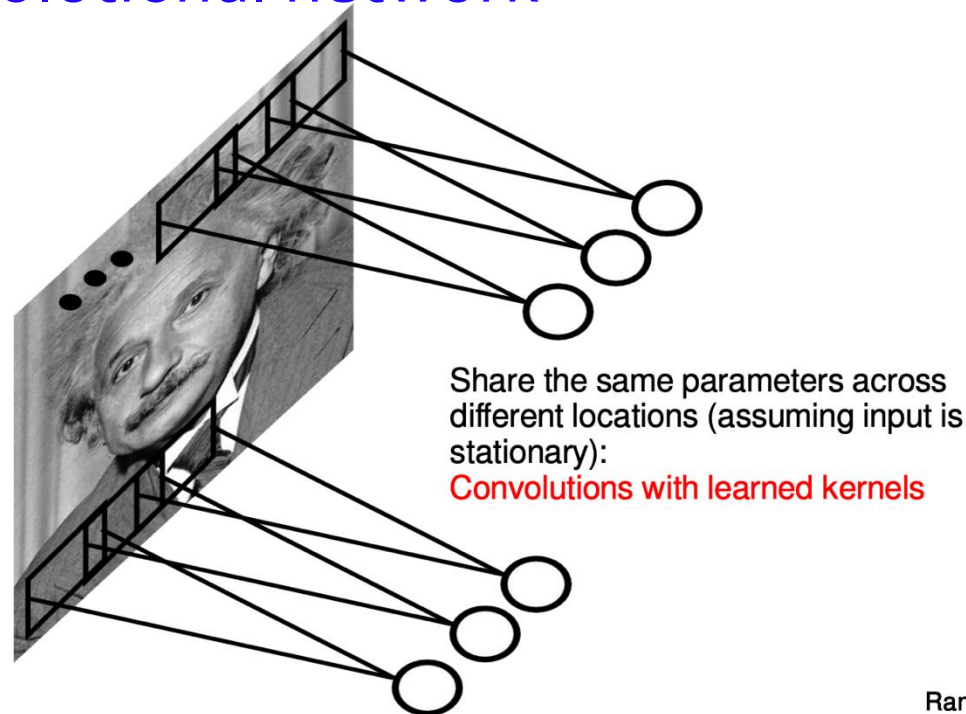


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

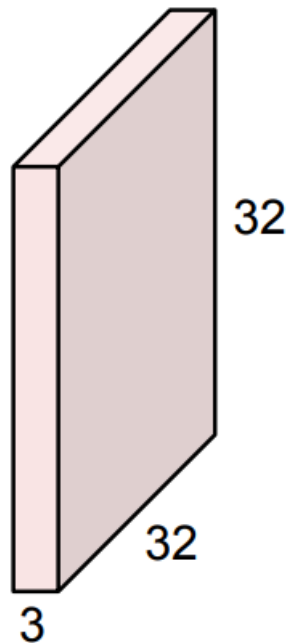
Convolutional Neural Net

- **Assumption (Idea):** statistics are similar at different locations (Lecun 1998)
- Connect each hidden unit to a small input patch and share the weight across space
- This is called a **convolution layer** and the network is a **convolutional network**



Convolution Layer

32x32x3 image



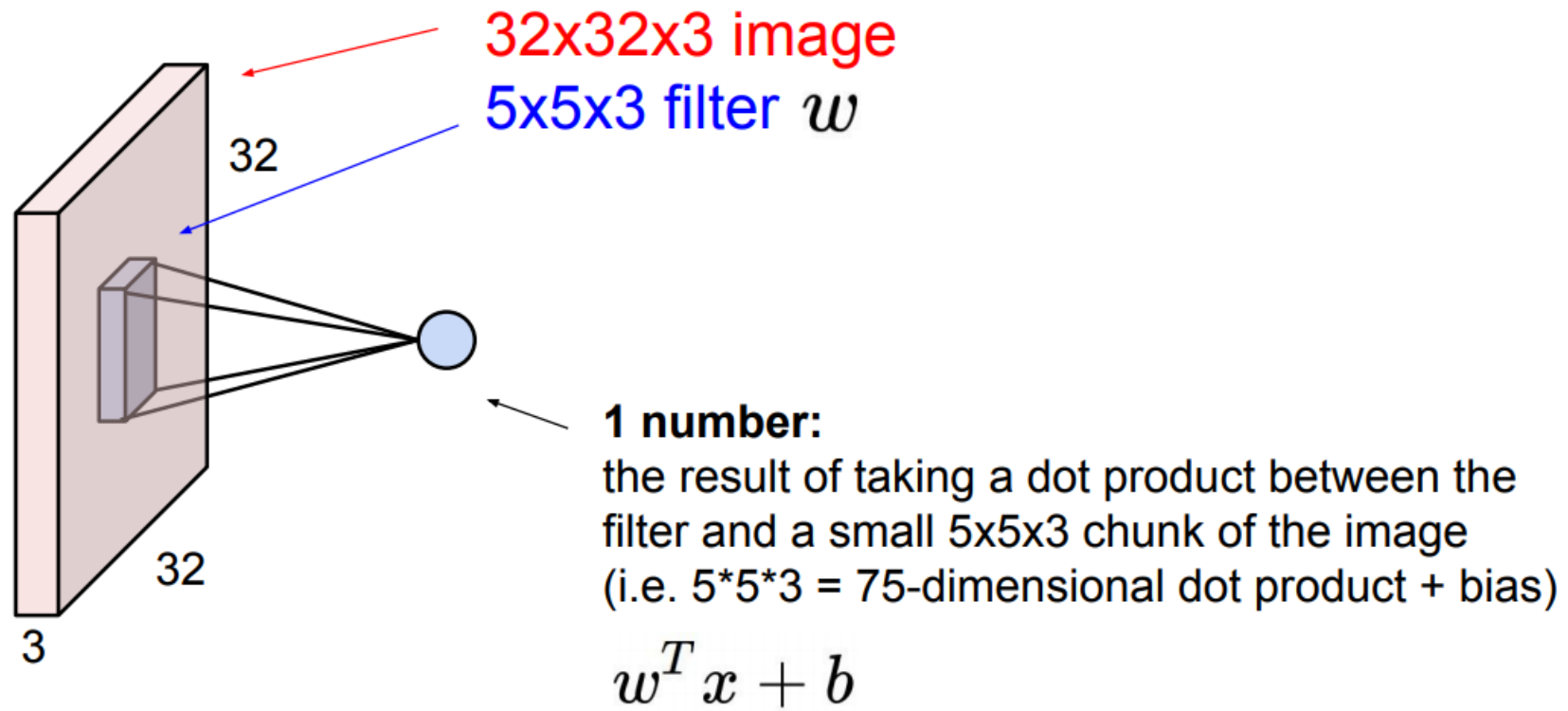
Filters always extend the full depth of the input volume

5x5x3 filter

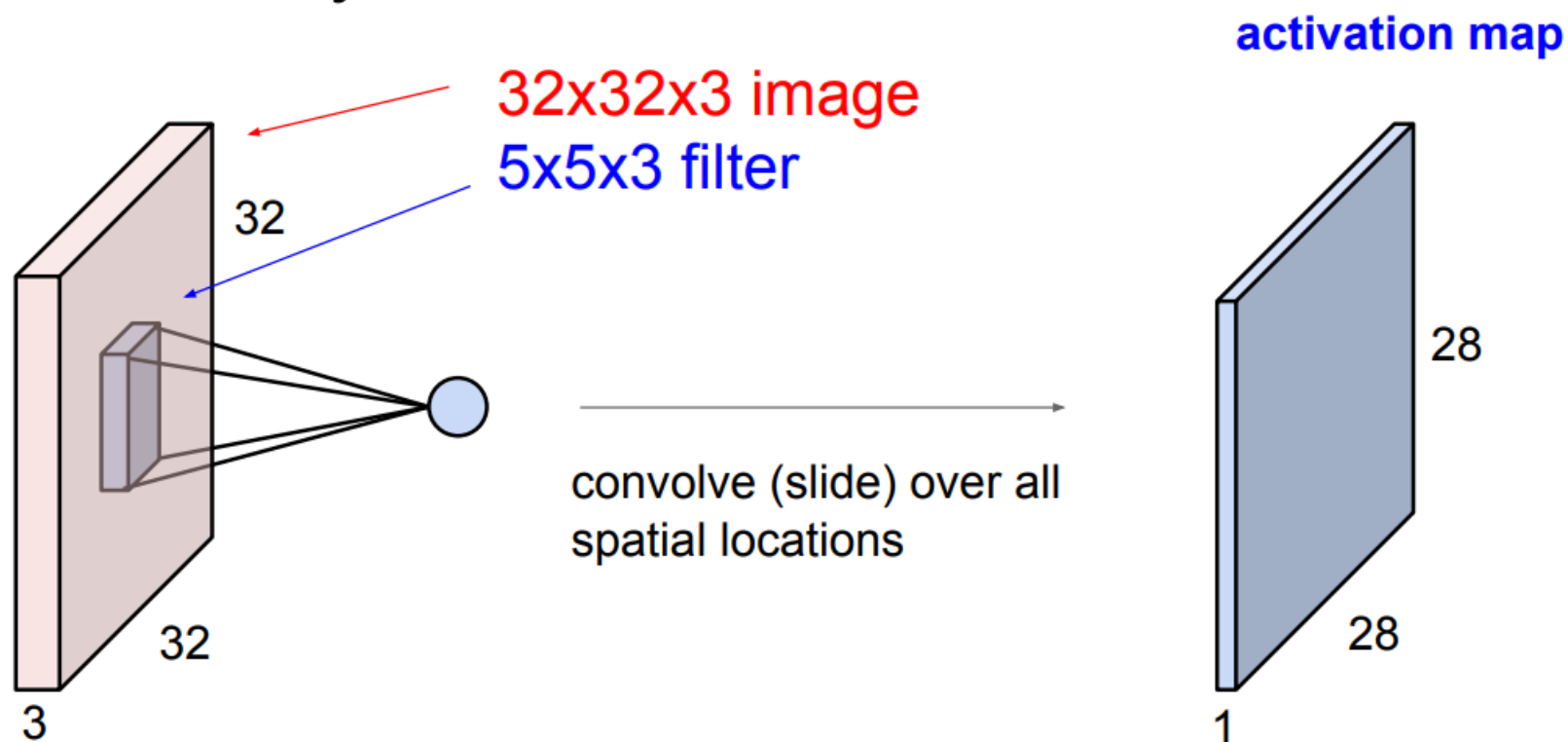


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

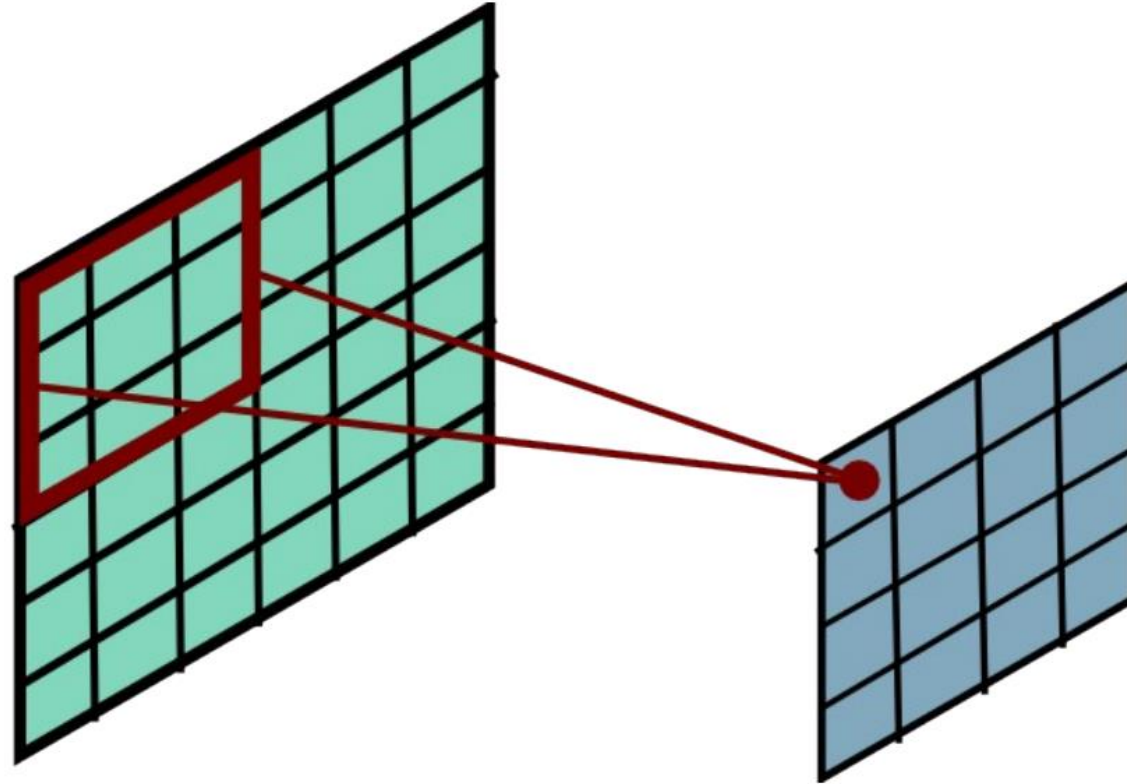
Convolution Layer



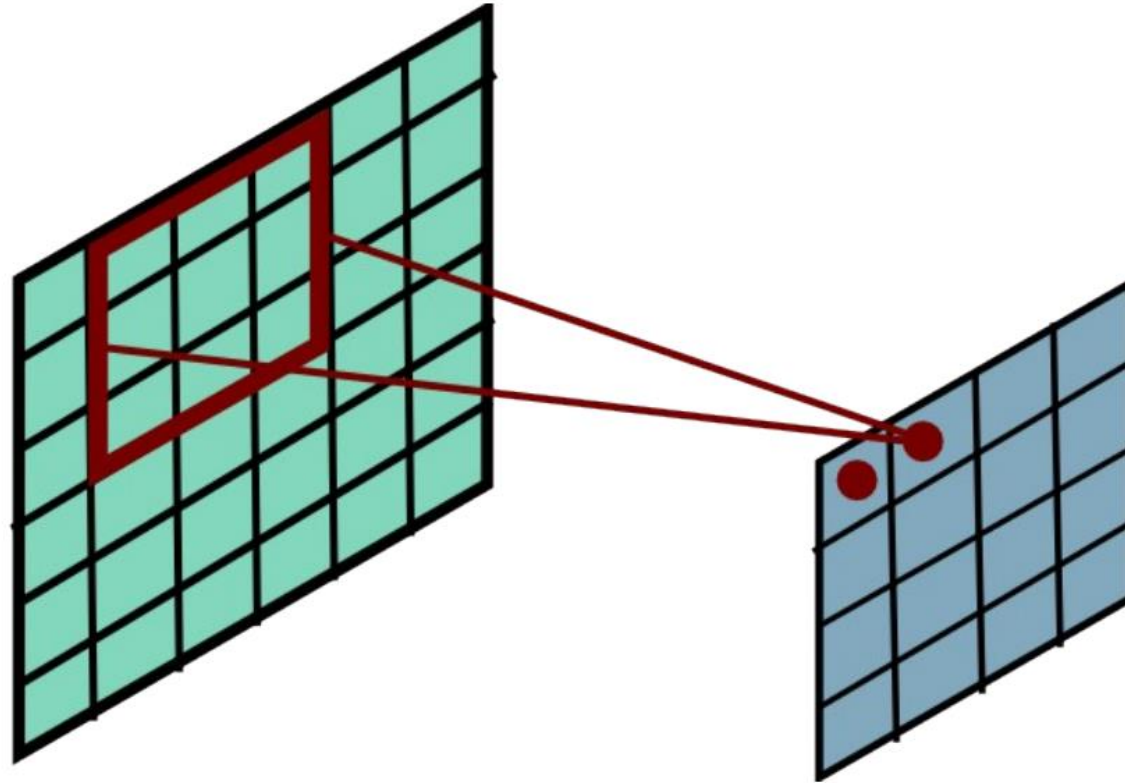
Convolution Layer



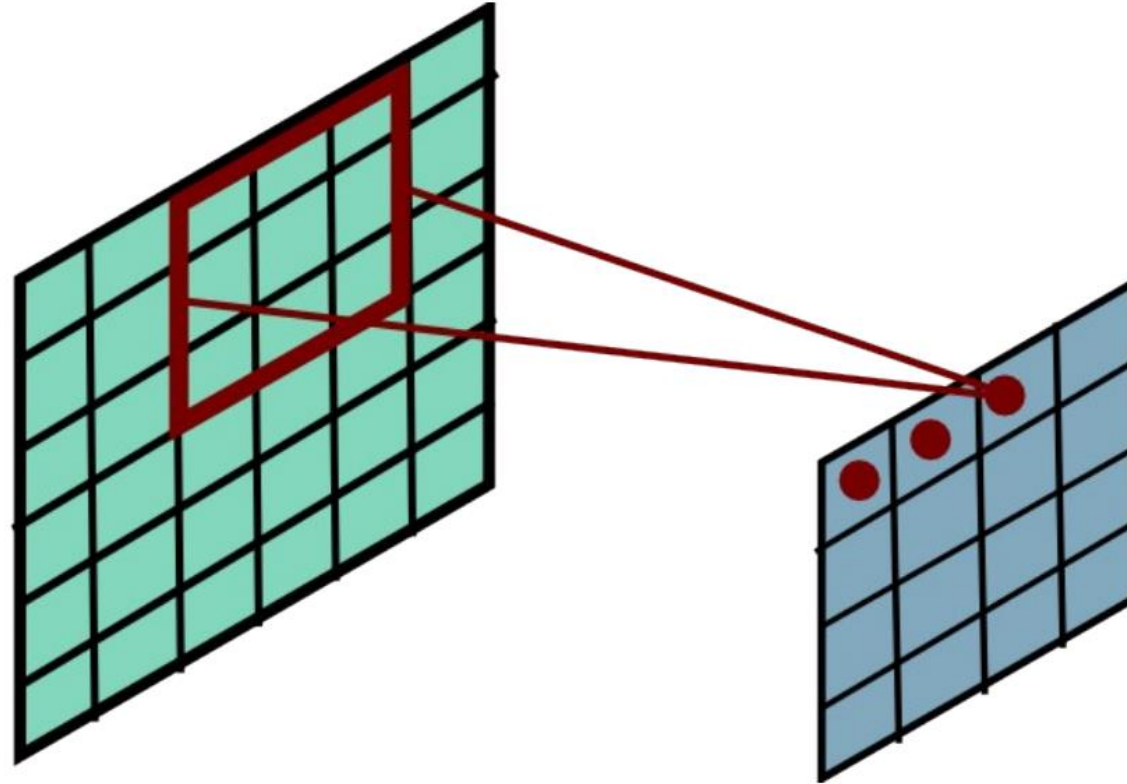
Convolutional Layer



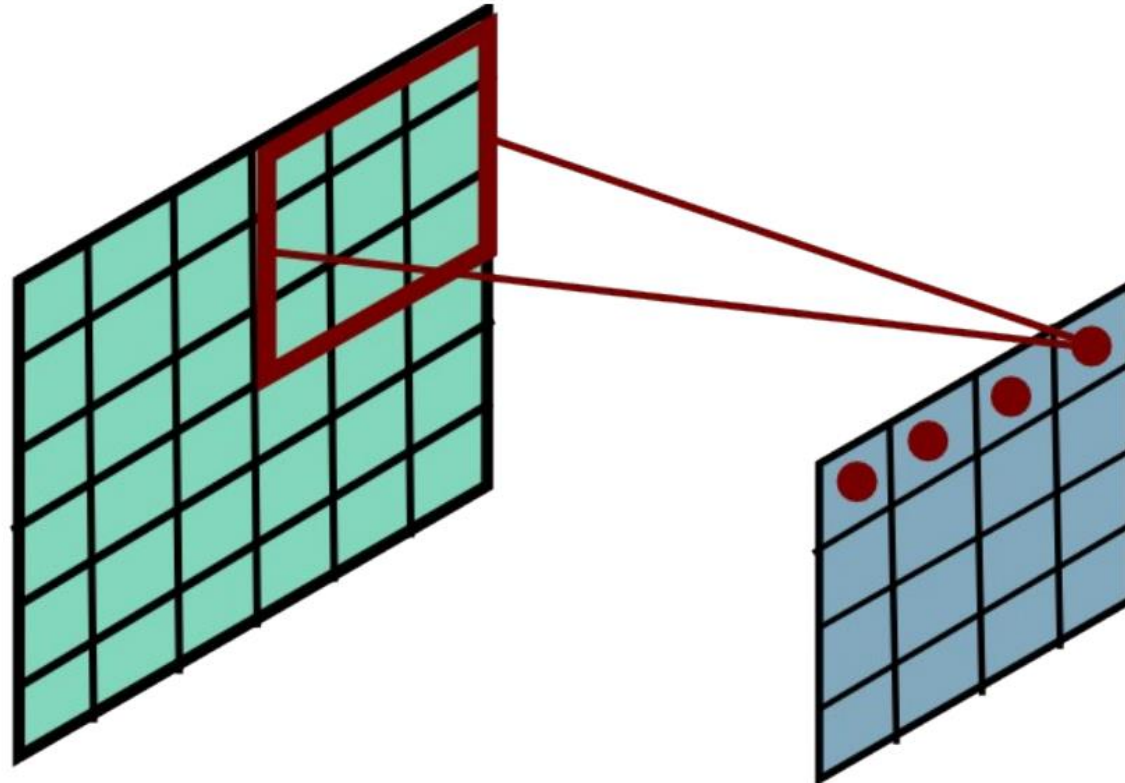
Convolutional Layer



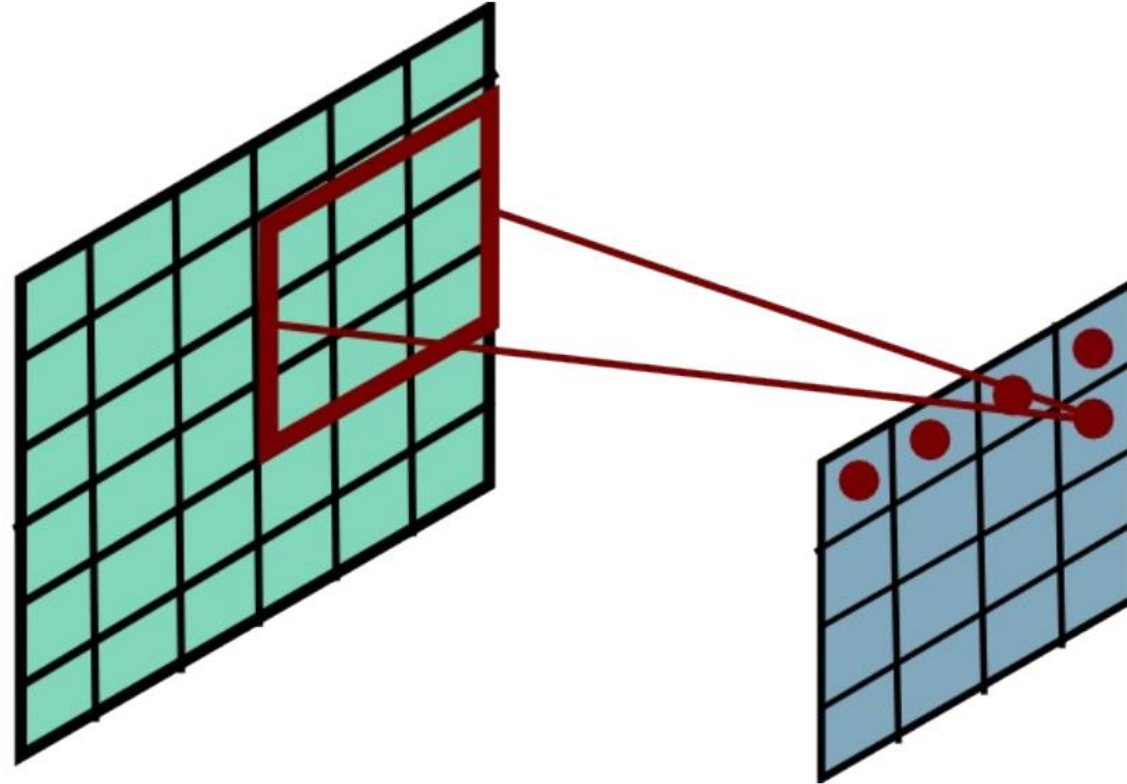
Convolutional Layer



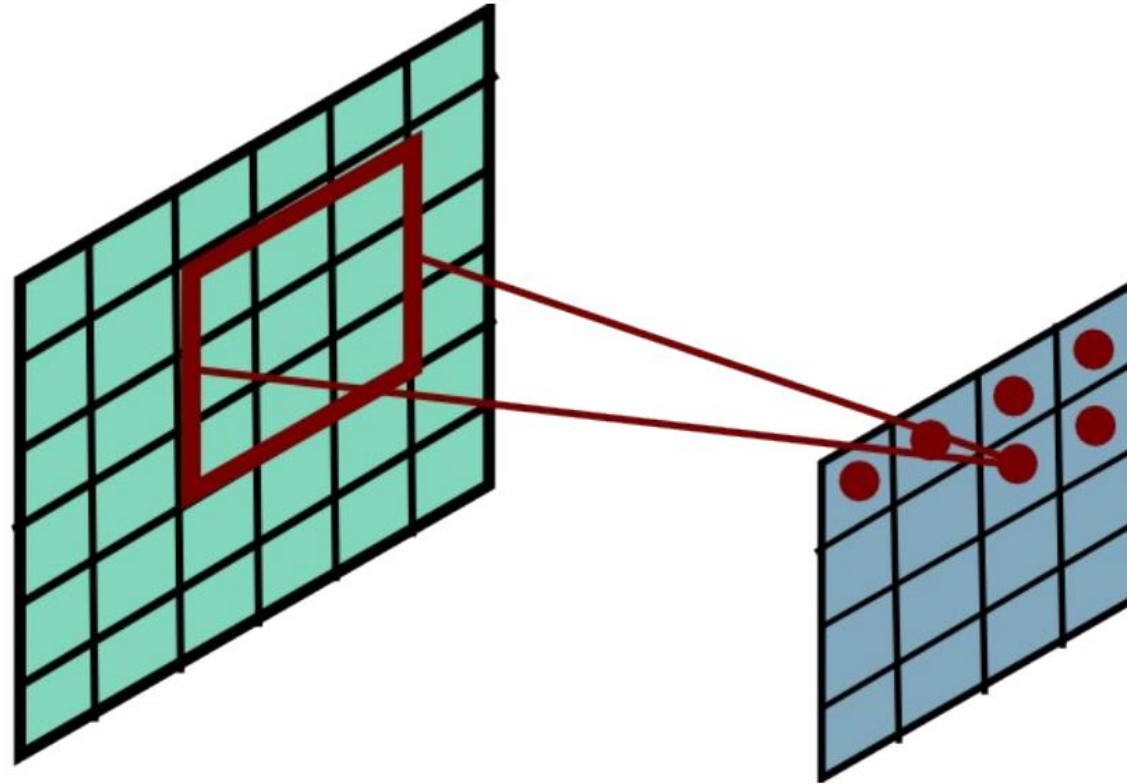
Convolutional Layer



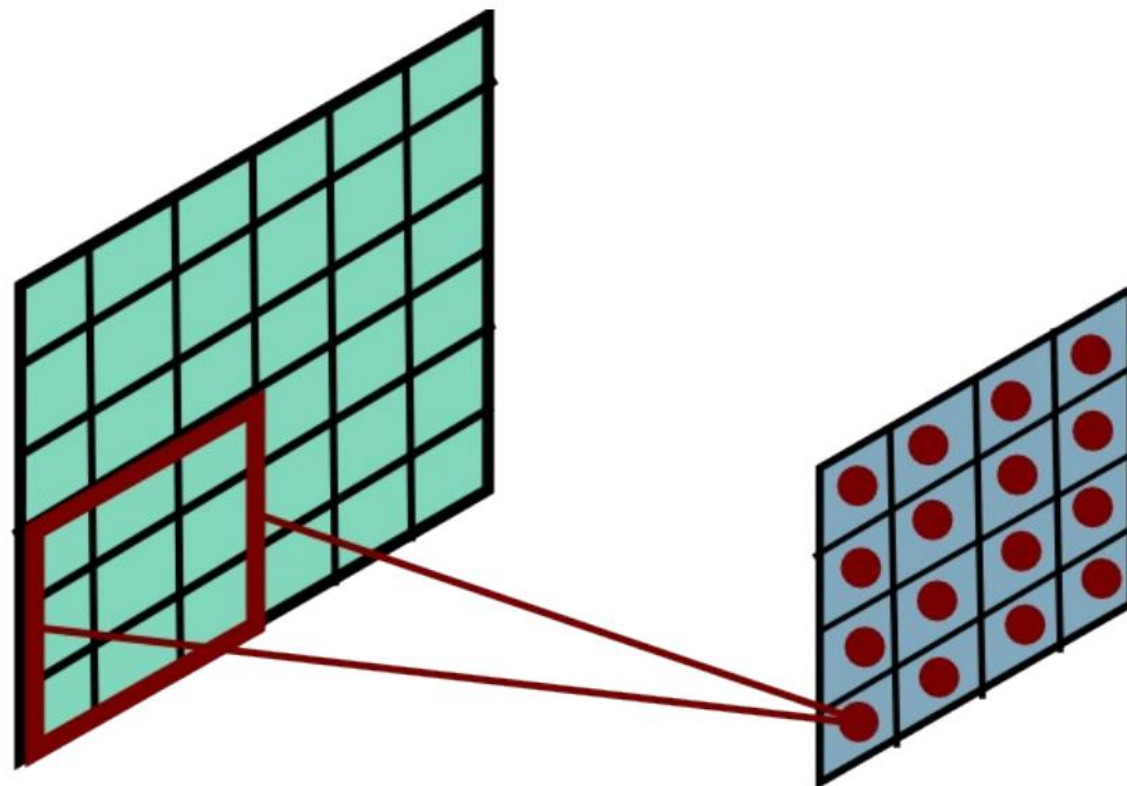
Convolutional Layer



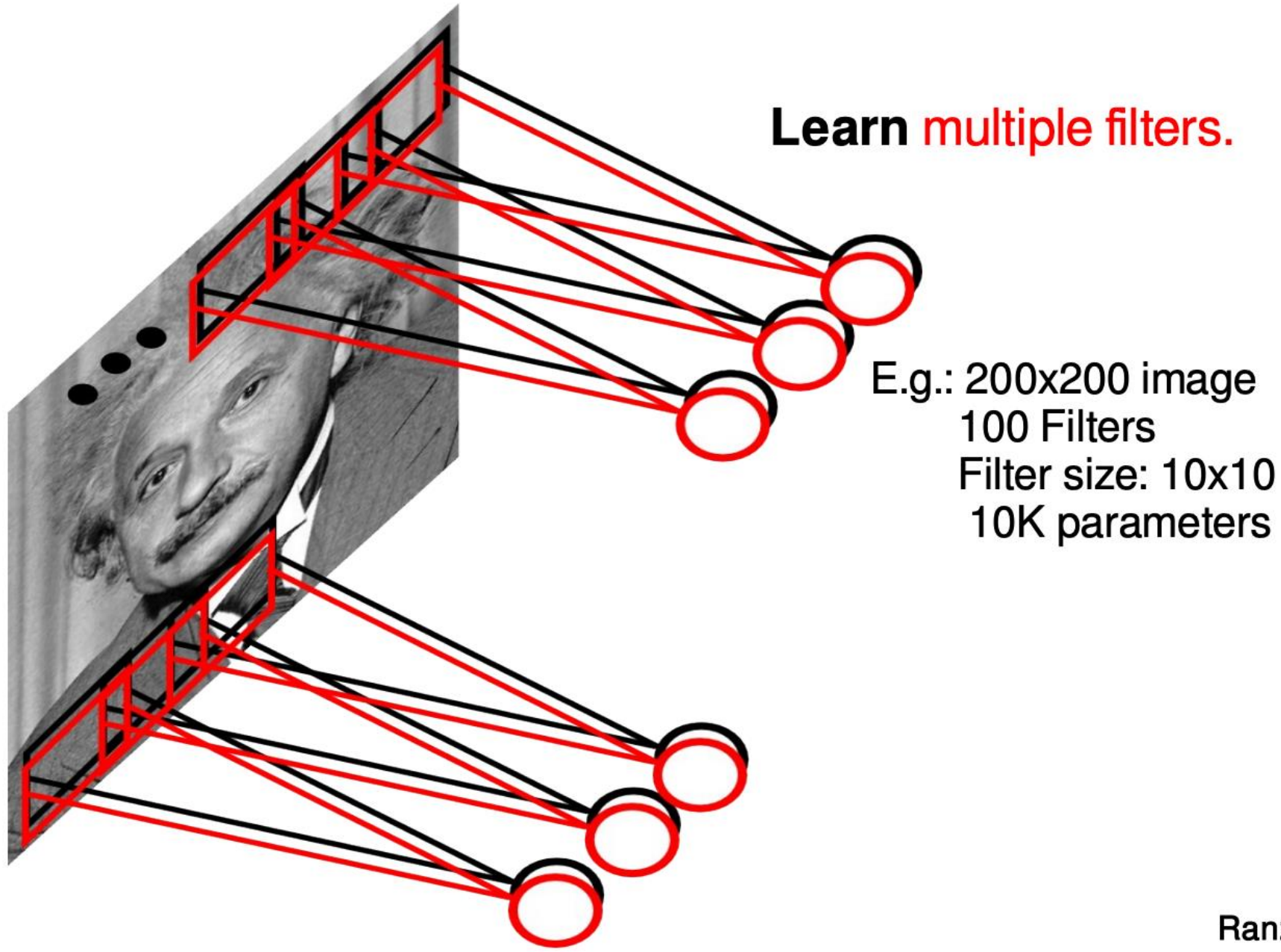
Convolutional Layer



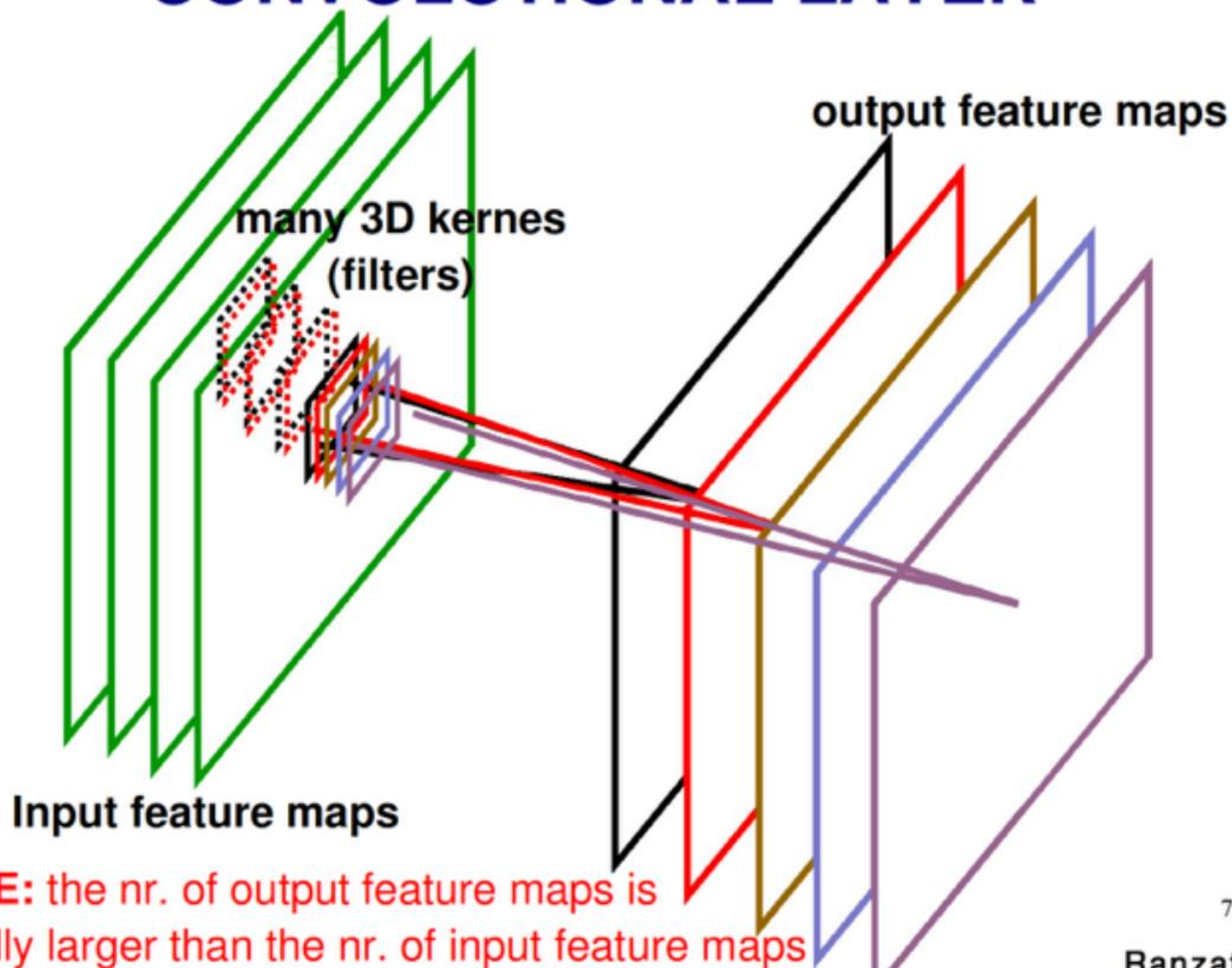
Convolutional Layer



Convolutional Layer



CONVOLUTIONAL LAYER

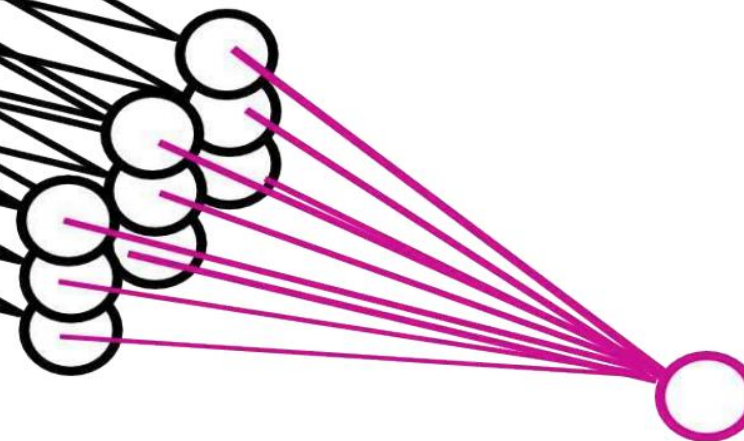


NOTE: the nr. of output feature maps is usually larger than the nr. of input feature maps

Max Pooling Layer



By “**pooling**” (e.g., taking max) filter responses at different locations we gain robustness to the exact spatial location of features.



Why Called as “Pooling”?

- Choose a number from a pool of numbers
- Other choices of “pooling” operation?
 - **Max Pooling**: return the maximal argument
 - **Average Pooling**: return the average of the arguments
 - Other types of pooling exist.

Example: Max Pooling

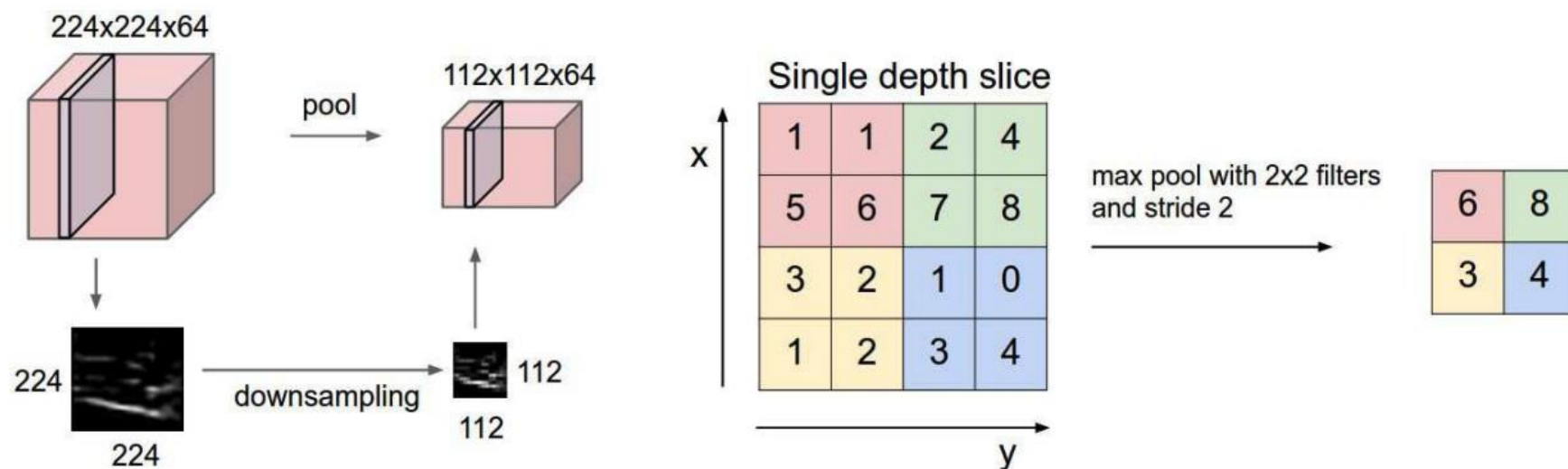
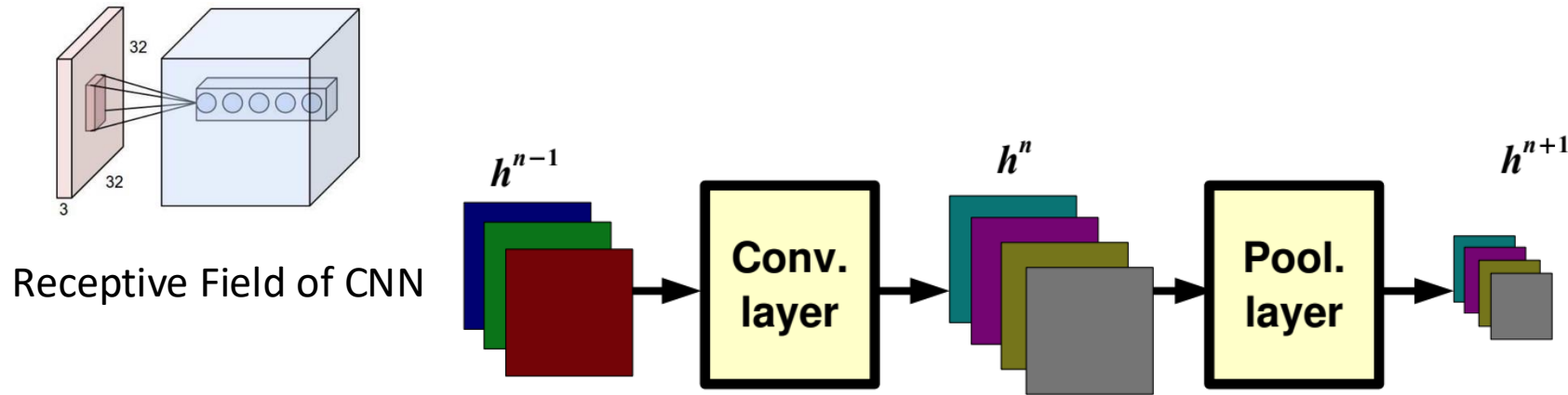


Figure: **Left:** Pooling, **right:** max pooling example

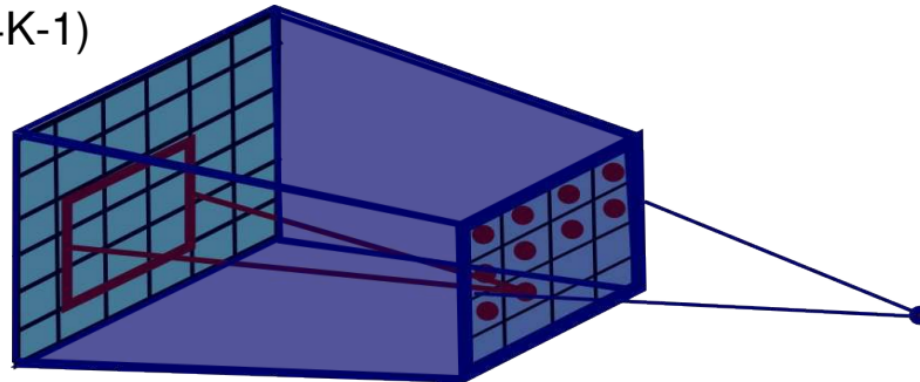
Hyperparameters of a pooling layer:

- The spatial extent F
- The stride

Pooling Layer Increase Receptive Field Size



If convolutional filters have size $K \times K$ and stride 1, and pooling layer has pools of size $P \times P$, then each unit in the pooling layer depends upon a patch (at the input of the preceding conv. layer) of size: $(P+K-1) \times (P+K-1)$



Pooling Layer May Remove Unnecessary Noise

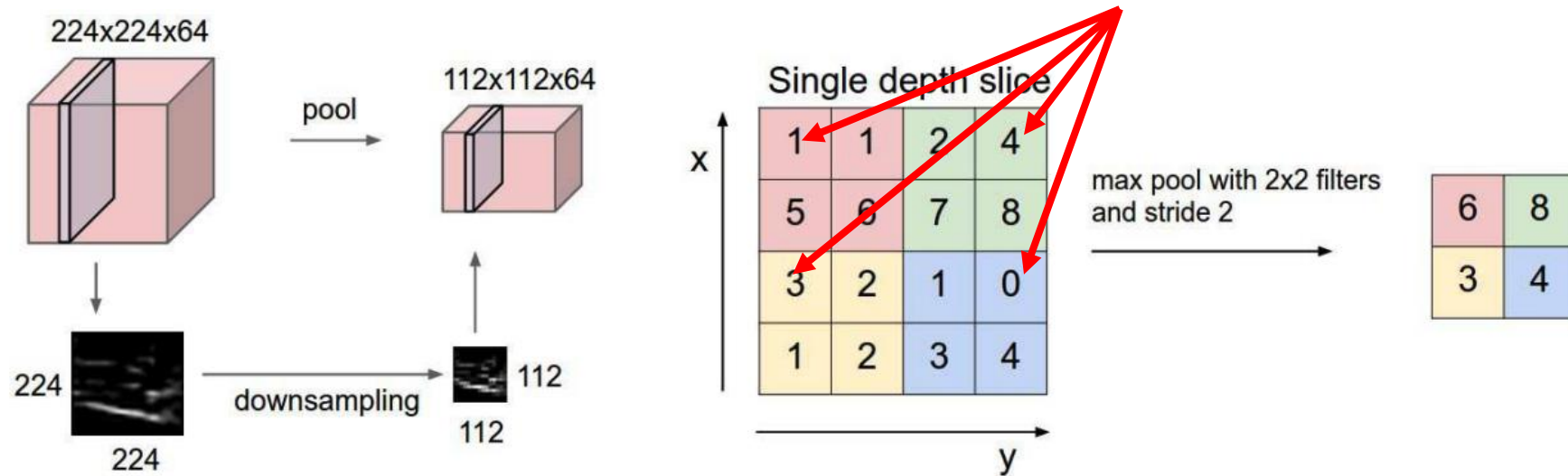


Figure: **Left:** Pooling, **right:** max pooling example

Hyperparameters of a pooling layer:

- The spatial extent F
- The stride

Multi-Layer Perceptron (MLP)

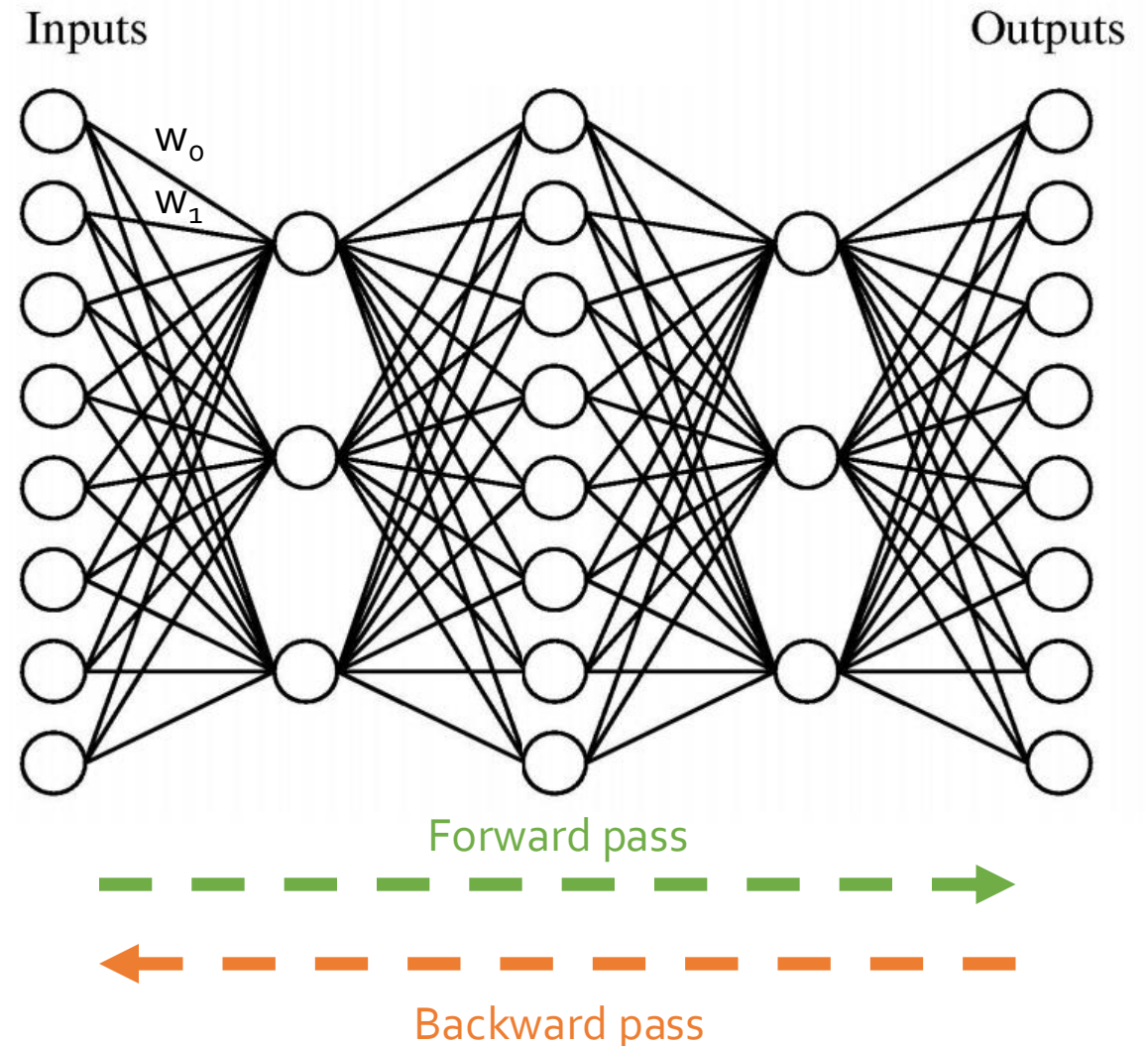
- Inference and learning

- Forward pass:

Left to right, each hidden layer in turn

- Backward pass:

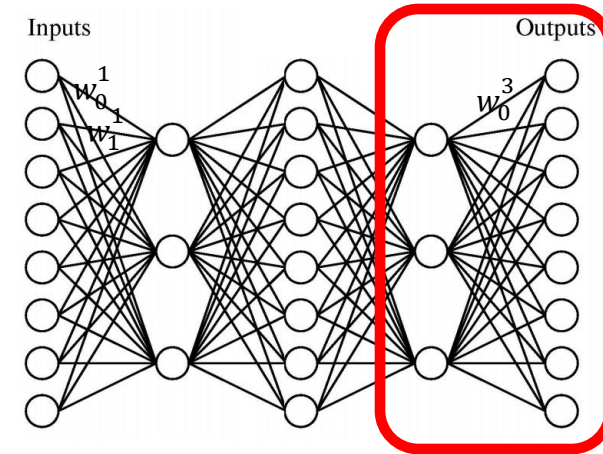
Right to left, propagating error to update the weight



Training a neural network (MLP) by the backpropagation

- Gradient descent (GD) algorithm

- Set $W = 0$ ($W = \{w^1, w^2, \dots\}$)
- For each sample (m samples total)
 - Perform forward propagation
 - To compute the loss: $loss(W)$
 - For each layer l
 - Compute ΔW^l Start from the **output** layer
 - Compute gradient of a node V_k with parents $U_1, U_2, \dots$
 - Compute the gradient of w_i^k : i^{th} node in the k^{th} layer: $\nabla_{w^l} loss(W)$
 - $\Delta W^l := \Delta W^l + \nabla_{w^l} loss(W)$
- Update the parameters
 - $W^l := W^l - \lambda \left(\frac{1}{m} \Delta W^l \right)$



Training deep networks in practice

- **Vanilla** Stochastic Gradient Descent (SGD)

- $W^{l,j} := W^{l,j} - \lambda \left(\frac{1}{k} \Delta W^{l,j} \right)$

- To get better stability and faster convergence

- Use momentum

- Moving average of weight update

$$V_t^{l,j} := \mu V_t^{l,j} - \lambda \left(\frac{1}{k} \Delta W_t^{l,j} \right)$$

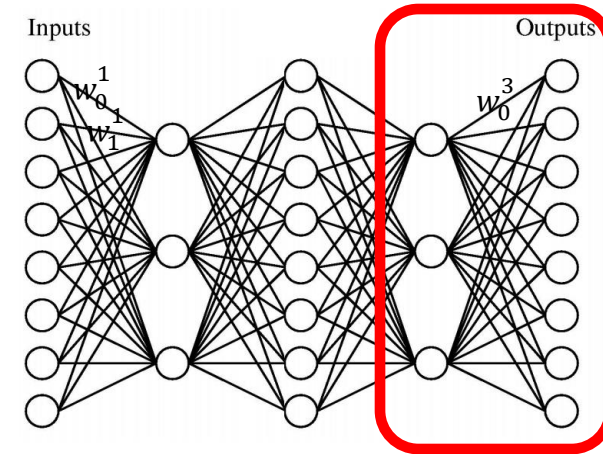
$$W_{t+1}^{l,j} = W_t^{l,j} + V_{t+1}^{l,j}$$

- Changing the weight slowly → less abrupt gradient descent → more stable training

Training the a neural network (MLP) by the backpropagation

- Gradient descent (GD) algorithm

- Set $W = 0$ ($W = \{w^1, w^2, \dots\}$)
- For each sample (m samples total)
 - Perform forward propagation
 - To compute the loss: $loss(W)$
 - For each layer l
 - Compute ΔW^l Start from the **output** layer
 - Compute gradient of a node V_k with parents $U_1, U_2, \dots$
 - Compute the gradient of w_i^k : i^{th} node in the k^{th} layer: $\nabla_{w^l} loss(W)$
 - $\Delta W^l := \Delta W^l + \nabla_{w^l} loss(W)$
- Update the parameters
 - $W^l := W^l - \lambda \left(\frac{1}{m} \Delta W^l \right)$



What if your data is big: either m is big or each sample is big?

Computationally very expensive!

Magic of learning rate

- Learning rate (LR) determines the details of training
- Basic LR scheme
 - LR decreasing scheduler
 - a.k.a. **optimizer**
 - So, it's not an actual optimizer
 - Bottou (Power scheduler)
 - At epoch t , rate: r
 - $\gamma_t = \gamma_0 \left(1 + \frac{t}{r}\right)^{-c}$
 - Exponential
 - $\gamma_t = \gamma_0 \times 10^{-\frac{t}{r}}$
 - Manual
 - If training gets plateau, lower the LR by half
- Dimension wise moving-average
 - Ada-series (AdaGrad, AdaDelta, Adam...)

Training the MLP (neural network) by the backprop.

- Gradient descent (GD) algorithm

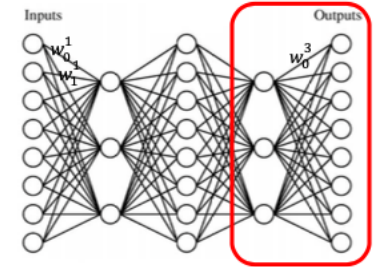
- Set $W = 0$ ($W = \{w^1, w^2, \dots\}$)
- For each sample (m samples total)
 - Perform forward propagation
 - To compute the loss: $loss(W)$
 - For each layer l
 - Compute ΔW^l Start from the **output** layer
 - Compute gradient of a node V_k with parents $U_1, U_2, \dots$
 - Compute the gradient of w_i^k : i^{th} node in the k^{th} layer: $\nabla_{w_i^k} loss(W)$
 - $\Delta W^l := \Delta W^l + \nabla_{w_i^k} loss(W)$

- Update the parameters

- $W^l := W^l - \lambda \left(\frac{1}{m} \Delta W^l \right)$

What if your data is big: either m is big or each sample is big?

Computationally very expensive!



“Ada-” LR schemes (Optimizers)

- AdaGrad
 - Dimension-wise moving average

$$\begin{bmatrix} \gamma_t^0 \\ \gamma_t^1 \\ \gamma_t^2 \\ \dots \\ \gamma_t^n \end{bmatrix} = \begin{bmatrix} \frac{\gamma_{t-1}^0}{\sqrt{\sum_t (\gamma_t^0)^2}} \\ \frac{\gamma_{t-1}^1}{\sqrt{\sum_t (\gamma_t^1)^2}} \\ \frac{\gamma_{t-1}^2}{\sqrt{\sum_t (\gamma_t^2)^2}} \\ \dots \\ \frac{\gamma_{t-1}^n}{\sqrt{\sum_t (\gamma_t^n)^2}} \end{bmatrix}$$

“Ada-” LR schemes (Optimizers)

- RMSProp
 - Improve AdaGrad

$$\begin{bmatrix} \gamma_t^0 \\ \gamma_t^1 \\ \gamma_t^2 \\ \dots \\ \gamma_t^n \end{bmatrix} = \begin{bmatrix} \frac{\exp \gamma_{t-1}^0}{\sqrt{\sum_t \exp(\gamma_t^0)^2}} \\ \frac{\exp \gamma_{t-1}^1}{\sqrt{\sum_t \exp(\gamma_t^1)^2}} \\ \frac{\exp \gamma_{t-1}^2}{\sqrt{\sum_t \exp(\gamma_t^2)^2}} \\ \dots \\ \frac{\exp \gamma_{t-1}^n}{\sqrt{\sum_t \exp(\gamma_t^n)^2}} \end{bmatrix}$$

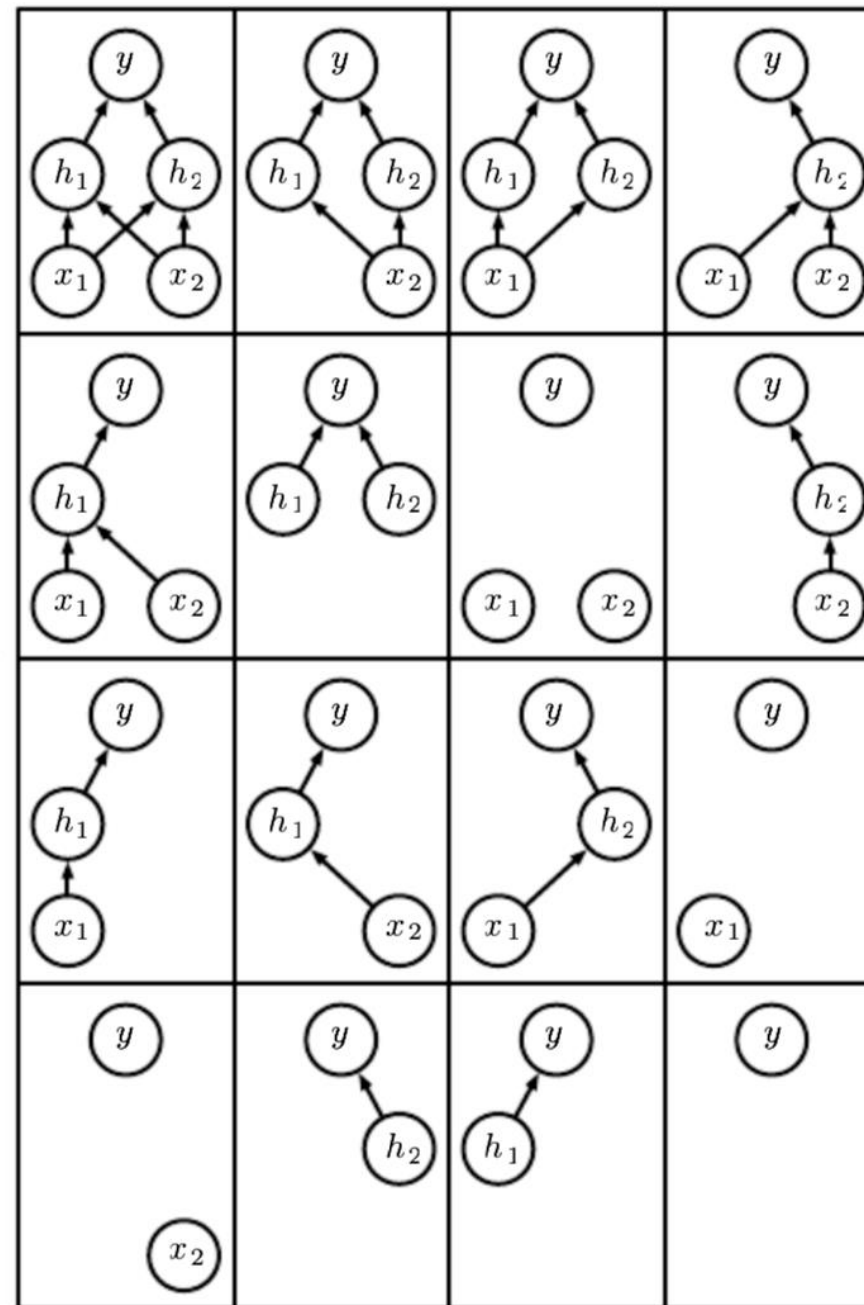
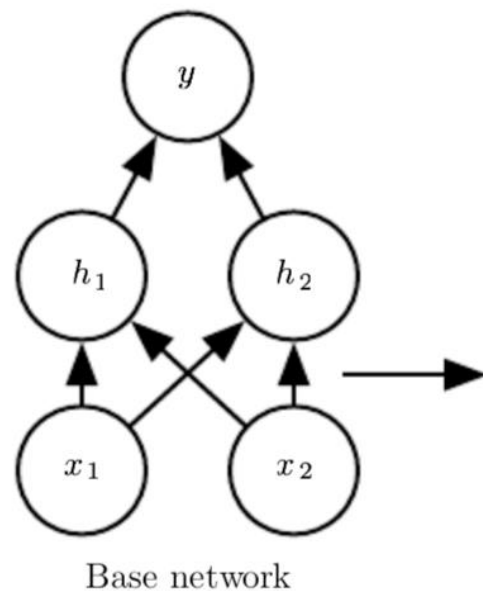
“Ada-” LR schemes (Optimizers)

- Adam
 - **Adaptive Moment**
 - Improvement upon RMSProp

$$\begin{bmatrix} \gamma_t^0 \\ \gamma_t^1 \\ \gamma_t^2 \\ \dots \\ \gamma_t^n \end{bmatrix} = \begin{bmatrix} \gamma_{t-1}^0 - \epsilon^0 \frac{\widetilde{s^0}}{\sqrt{\widetilde{r^0} + \delta^0}} \\ \gamma_{t-1}^1 - \epsilon^1 \frac{\widetilde{s^1}}{\sqrt{\widetilde{r^1} + \delta^1}} \\ \gamma_{t-1}^2 - \epsilon^1 \frac{\widetilde{s^2}}{\sqrt{\widetilde{r^2} + \delta^2}} \\ \dots \\ \gamma_{t-1}^n - \epsilon^n \frac{\widetilde{s^n}}{\sqrt{\widetilde{r^n} + \delta^n}} \end{bmatrix}$$

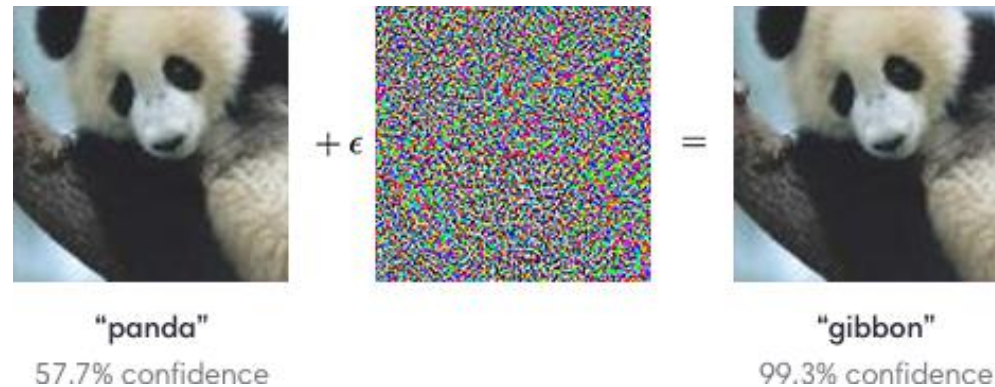
- s: biased first-order moment estimate
- r: biased second-order moment estimate

DropOut and Ensemble



Ensemble of subnetworks

Noise Robustness



- Method 1: Add noise in the training samples
- Method 2: Inject noise at the output target (**Label smoothing**)
 - $0 \rightarrow \frac{\epsilon}{k-1}$
 - $1 \rightarrow 1 - \epsilon$
 - ϵ : small value
 - k : number of classes

Early Stopping

- Going far makes network converge to a local minimum
- Better to *less* converge to the local minimum
- Stop before the network completely converges

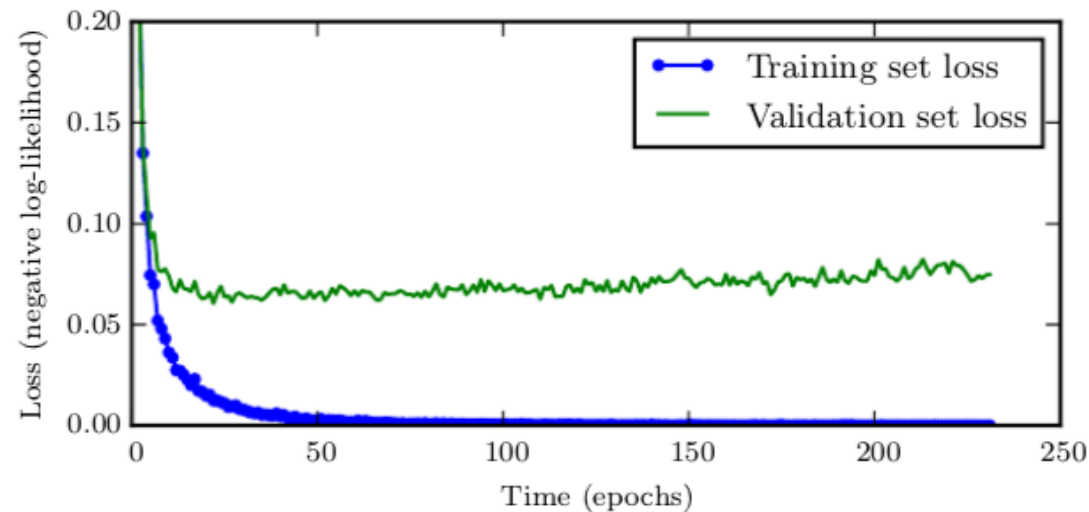


Figure 7.3: Learning curves showing how the negative log-likelihood loss changes over time (indicated as number of training iterations over the dataset, or **epochs**). In this example, we train a maxout network on MNIST. Observe that the training objective decreases consistently over time, but the validation set average loss eventually begins to increase again, forming an asymmetric U-shaped curve.

Summary

- CNN consists of
 - Convolutional layers
 - (max) Pooling layers
 - MLP (fully connected) layers